DataInsights - An Explainable Retrieval-Augmented Natural Language to SQL System Using Large Language Model

By Navya Bala Sathyamurthi (G00901982)

Abstract

DataInsights is a Natural Language to SQL (NL→SQL) system that enables users to query relational databases using natural language. It is a Large Language Model (LLM) application that converts the Natural Language Questions to SQL query, executes the query over a database and return the results. In addition, DataInsights explains the results back in natural language to the user. The System is incorporated with Retrieval-Augmented Generation (RAG) which is using FAISS-based vector store to improve the SQL accuracy by providing relevant schema and relationships as context to guide generation and reduce hallucinations. The generated SQL query is executed over a real-world E-commerce database named **Brazilian Olist E-commerce Dataset**. Additionally, a self-correcting mechanism addresses SQL execution errors, and the system generates natural language explanations for the results.

---

1. Introduction

Structured data is widely used across various domains such as e-commerce, banking, medical, education etc, yet accessing it requires knowledge of SQL, which can be challenging for non-technical users. DataInsights, a Natural Language to SQL (NL→SQL) system aims to bridge this gap by enabling users to query databases using everyday natural language.

Despite recent progress using large language models, the main challenges are:

- Lack of schema awareness
- Incorrect joins across tables
- Hallucinated columns
- Lack of interpretability

This project addresses these challenges by integrating Retrieval-Augmented Generation (RAG) with LLM-based SQL generation. The system retrieves relevant database context before query generation, enabling more accurate SQL outputs. In addition, it generates natural language explanations to make results understandable for users without SQL expertise.

---

2. Related Work

Natural language to SQL conversion were done by using rule-based and template-based approaches. Recent NL→SQL systems use transformer models but they lack knowledge in database schemas and may generate incorrect or non-executable queries.

This project uses RAG to improve the LLM performance by providing the context. Many existing systems just produces query. But My application explains the query results back in natural language so that it can be easily understandable by the non-technical users. This project combines schema-aware retrieval, self-correcting loop and natural language explanation.

---

3. Data

3.1 Dataset

The system uses **Brazilian Olist E-commerce Dataset**, from Kaggle which is a real-world dataset that contains e-commerce transaction data from Brazil.

3.2 Tables Used

- customers
- orders
- order_items
- order_payments
- order_reviews
- products
- sellers
- geolocation
- product_category_name_translation

3.3 Preprocessing

- Deleted some of the tables to reduce the size of the database
- Extracted schema (tables and columns)
- Defined relationships between tables
- Created text documents for each table
- Converted into Lang Chain `Document` objects
- Generated data into embeddings for retrieval

---

4. Methods

4.1 System Architecture

DataInsights follows a multi-stage pipeline:

1. User inputs a natural language question
2. Relevant context is retrieved using a FAISS vector store
3. User question and context are passed to the LLM
4. LLM generates SQL query
5. Query is validated and corrected (if necessary)
6. Query is executed on the database and results obtained
7. Results passed on to LLM for generation of Natural language explanation
8. Results and explanation are displayed in the application

4.2 Retrieval-Augmented Generation (RAG)

RAG which is an AI framework helps improve the LLM accuracy by providing specific information while answering the complex questions. DataInsights uses **FAISS** (Facebook AI Similarity Search) to retrieve relevant context based on semantic similarity, including:

- Table schemas
- Column descriptions
- Relationships

This ensures that the LLM generates SQL based on accurate and relevant information.

4.3 SQL Generation

A prompt template combines:

- Retrieved context
- User question

The LLM GPT-4.1-mini, a lightweight large language model based on the transformer architecture. The model is optimized for low-latency and cost-efficient inference while maintaining strong performance on structured generation tasks such as SQL and code generation. Although the exact number of parameters is not publicly disclosed, GPT-4.1-mini belongs to the GPT-4.1 family of autoregressive transformer models, which are trained on large-scale text corpora using next-token prediction. The model leverages self-attention mechanisms to capture contextual relationships within input sequences and generates SQL queries following the schema constraints and uses relationships to perform correct joins and aggregation operations.

4.4 Self-Correcting Mechanism

If a generated SQL query fails during execution, the error message is sent back to the model along with the wrong SQL query generated, user question and the context and asking the model to fix it. This shows system robustness and the agentic behavior.

4.5 Explanation Generation

DataInsights generates natural language explanations for the generated SQL query output. This improves interpretability for non-technical users.

4.6 User Interface

Streamlit is used for user interface which will show the following:

- User input question
- Query results
- Natural language explanations
- Retrieved RAG context (for transparency)

---

5. Experiments & Results

5.1 Evaluation Metrics

DataInsights is evaluated by checking the query results the LLM generated with the query results that was manually executed by me in the DB Browser for SQLite. Execution-based accuracy was checked to measure the SQL correctness. The explanation quality was manually qualitatively evaluated based on clarity and correctness.

5.2 Results

User Question:

Give me id of customers who bought the most expensive item
Generated SQL:

```
SELECT o.customer_id
FROM orders o
JOIN order_items oi ON o.order_id = oi.order_id
ORDER BY oi.price DESC
LIMIT 1
```

Explanation:
The customer with the ID 'c6e2731c5b391845f6800c97401a43a9' bought the most expensive item.

5.3 Observations

- RAG significantly improves SQL accuracy
- Relationship-aware retrieval improves join correctness
- Explanation generation improves usability and transparency

---

6. Contributions

I have accomplished the following in this project:

- Development of a RAG-based NL→SQL system using FAISS
- Integration of schema-aware retrieval
- Implementation of a self-correcting SQL generation mechanism
- Generation of natural language explanations for improved interpretability
- Deployment of an interactive user interface for real-time querying

---

7. Limitations

- Not able to handle very large number of rows as results due to the token limit of the model.
- No evaluation dataset
- No automated metric for explanation quality
- Limited handling of semantically incorrect (but valid) queries

---

8. Future Work

- Semantic correction for incorrect or empty results
- Automated evaluation of explanation quality
- Multi-database support
- Fine-tuned models for SQL generation

---

9. Citations & Acknowledgements

Dataset

- Brazilian Olist E-commerce Dataset (Kaggle)

Tools & Frameworks

- LangChain
- FAISS
- OpenAI API
- Streamlit

Concepts

- Retrieval-Augmented Generation (RAG)
- Natural Language Processing (NLP)
- Explainable AI (XAI)